04. Transformers

Overview

Transformers replace recurrence with self-attention so every token attends to every other token in parallel. We walk through multi-head attention, positional encoding, feed-forward blocks, layer normalization, and the encoder/decoder stack. Complexity, masking, and pre-norm vs post-norm choices are tied directly to implementation details in PyTorch.
Lab: Lab 04.
Prior modules: Module 02, **Module 03**.

Contents

Module track

This module builds on **Module 02**, **Module 03**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

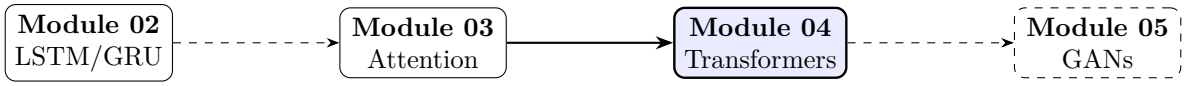

Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

Recap from Module 03

Attention forms a context vector as a softmax-weighted sum of encoder states. Self-attention applies the same mechanism within one sequence, enabling parallel training without recurrence.

1 Self-Attention Mechanism

Transformers [6] replace recurrence with self-attention over a sequence. For input matrix $\mathbf{X} \in \mathbb{R}^{T \times d}$, projections yield queries, keys, values:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V. \tag{1}$$

2 Scaled Dot-Product Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}. \tag{2}$$

Scaling by $\sqrt{d_k}$ prevents dot products from growing large, which would push softmax into saturated regions with tiny gradients. This generalizes Bahdanau scoring in **Module 03** (additive attention scores).

Note. Self-attention connects every position to every other in $O(T^2 d)$ compute, parallelizable on GPUs unlike **Module 01** (unrolled RNN diagram).

3 Multi-Head Attention

h heads attend in different subspaces:

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V), \tag{3}$$

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\,\mathbf{W}^O. \tag{4}$$

Default $h = 8$, $d_k = d/h$.

4 Transformer Block

Each layer applies MHA, residual connection, layer norm, position-wise FFN:

$$\mathbf{Z} = \text{LN}(\mathbf{X} + \text{MHA}(\mathbf{X}, \mathbf{X}, \mathbf{X})), \tag{5}$$

$$\mathbf{Y} = \text{LN}(\mathbf{Z} + \text{FFN}(\mathbf{Z})), \tag{6}$$

$$\text{FFN}(\mathbf{z}) = \max(0, \mathbf{z}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2. \tag{7}$$

Positional encodings inject order: $PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$.

Table 1: Base Transformer hyperparameters (Vaswani et al.).

Setting	**Value**
d_{model}	512
d_{ff}	2048
Layers	6 encoder + 6 decoder
Heads h	8
Dropout	0.1

5 Complexity Analysis

Self-attention layer complexity is $O(T^2 d)$ vs. $O(Td^2)$ per recurrent layer in **Module 02** (LSTM gate equations). For $T < d$, attention is competitive; long sequences motivate sparse/linear attention variants.

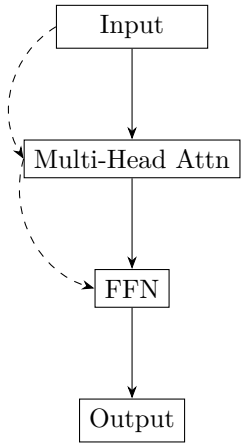

Figure 2: Residual connections around attention and FFN.

6 Downstream Impact

Transformers underpin BERT (**Module 09** (BERT architecture)) and GPT (**Module 10** (causal LM objective)). **Lab 04** builds a mini encoder stack implementing (2).

7 Positional Encoding Details

Sinusoidal encodings extrapolate to longer sequences than learned embeddings:

$$PE_{(pos,2i)} = \sin\Big(pos/10000^{2i/d}\Big)\,, \tag{8}$$

$$PE_{(pos,2i+1)} = \cos\Big(pos/10000^{2i/d}\Big)\,. \tag{9}$$

Relative position encodings (T5, RoPE) improve length generalization beyond fixed $T_{\max}$.

7.1 Encoder to Decoder Cross-Attention

Decoder queries attend encoder keys/values: $\mathbf{C} = \text{MHA}(\mathbf{Q}_{\text{dec}}, \mathbf{K}_{\text{enc}}, \mathbf{V}_{\text{enc}})$.

8 Extended Intuition

Transformers remove sequential recurrence: every token attends to every other token in one parallel pass (within a layer). Self-attention computes pairwise relevance, softmaxes into weights, and mixes value vectors. Multi-head attention runs several attention operations in parallel with different learned projections, then concatenates and projects back.

Positional information is not implicit (no step index in the recurrence), so models add sinusoidal or learned position embeddings to token vectors. Each encoder block is typically: multi-head self-attention, residual connection, layer norm, feed-forward MLP, residual, layer norm. Decoder blocks add masked self-attention (causal: token i cannot see $j > i$) and cross-attention to encoder outputs in seq2seq setups.

Complexity is $O(T^2 d)$ in sequence length for attention scores, which hurts long contexts but trains fast on GPUs because matmuls parallelize. Pre-norm (norm before sublayers) often trains more stably than post-norm at depth; most recent LLMs use pre-norm variants. BERT (**Module 09**) uses encoder-only stacks; GPT (**Module 10**) uses decoder-only stacks with causal masking.

8.1 Why Scale by $\sqrt{d_k}$

Dot products $\mathbf{q}_i^\top \mathbf{k}_j$ have variance scaling with d_k when entries are unit variance. Without division by $\sqrt{d_k}$, softmax inputs grow large for wide heads, pushing weights toward one-hot and shrinking gradients. The FFN sublayer is not an afterthought: two linear maps with inner dim $4d_{\text{model}}$ and GELU/ReLU carry most parameters and FLOPs per block. Pre-norm places LayerNorm before attention and FFN, so residual streams stay well-scaled deep into the stack (common in LLaMA-style configs). Rotary position embedding (RoPE) rotates Q/K by angle depending on distance; used in LLaMA instead of additive sinusoids on long contexts.

9 Numerical Walkthrough

Mini encoder layer: $T = 128$, $d_{\text{model}} = 256$, heads $H = 8$, $d_k = d_v = 32$, FFN inner dim $d_{ff} = 1024$, $B = 8$.

Input $\mathbf{X} \in \mathbb{R}^{B \times T \times 256}$. Per head, $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{B \times T \times 32}$. Attention scores $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top / \sqrt{32} \in \mathbb{R}^{B \times T \times T}$: $128^2 = 16384$ scores per head per batch item. Eight heads plus FFN: dominant FLOPs often come from the FFN ($4d_{\text{model}} d_{ff}$ per token per layer).

Memory for $\mathbf{S}$ in fp32: $B \cdot H \cdot T^2 \cdot 4$ bytes $= 8 \cdot 8 \cdot 16384 \cdot 4 \approx 4.2\,\text{MB}$ (small here; at $T = 4096$ it explodes). Training a 4-layer, 256-dim model on a tiny copy task: lr $= 10^{-4}$, warmup 400 steps, AdamW weight decay 0.01. Loss should drop below 0.1 nats within a few thousand steps if implementation is correct.

Per-token FFN cost: $2 \cdot T \cdot d_{\text{model}} \cdot d_{ff}$ multiply-adds (up and down projections). At $T = 128$, $d_{\text{model}} = 256$, $d_{ff} = 1024$: $2 \cdot 128 \cdot 256 \cdot 1024 \approx 67$M multiply-adds per layer forward, often exceeding attention FLOPs at moderate T. PyTorch 2.x `scaled_dot_product_attention` may dispatch to FlashAttention when shapes and dtype align; fall back to math backend if you see warnings.

Hand check for one head, $T = 3$, $d_k = 2$: if $\mathbf{q}_1^\top \mathbf{k}_1 / \sqrt{2} = 2$ and other scores are 0, then $\alpha_{1,1} = e^2/(e^2 + 2) \approx 0.88$. Stack depth $L = 6$, $d_{\text{model}} = 256$, $H = 8$: total MHA params per layer $\approx 4 \cdot 256^2 = 262$k; six layers ≈ 1.57M in attention alone. Label smoothing on next-token CE (0.1) in small classifiers reduces overconfident logits; less common in encoder-only classification but helps seq tagging heads.

10 PyTorch Implementation Notes

- `nn.TransformerEncoderLayer(d_model=256, nhead=8, dim_feedforward=1024, batch_first=True)`

Expects `(B, T, E)` when `batch_first=True` (PyTorch 2.x default in many examples).

- Causal mask: `nn.Transformer.generate_square_subsequent_mask(T)` or `torch.triu(torch.ones(T,T), diagonal=1).bool()`.
- Padding mask: `src_key_padding_mask` shape `(B, T)`, True where pad tokens sit.
- `nn.MultiheadAttention` with `need_weights=True` returns attention weights for visualization.
- Layer norm epsilon defaults to 10^{-5}; mismatch with pretrained checkpoints causes silent quality drops when fine-tuning.

```
layer = nn.TransformerEncoderLayer(256, 8, 1024, batch_first=True)
src_key_padding_mask = (tokens == PAD_ID)
out = layer(x, src_key_padding_mask=src_key_padding_mask)
```

11 Local GPU Constraints

A 4-layer, $d_{\text{model}} = 256$ transformer on $T = 128$, $B = 16$ fits in roughly 1.5 to 2 GB including Adam states. Doubling T quadruples attention memory; on 4 GB keep $T \leq 256$ for from-scratch training.

Use `torch.cuda.amp.autocast` and `GradScaler` for larger effective models. Gradient checkpointing (`torch.utils.checkpoint`) trades compute for memory inside deep stacks. FlashAttention kernels (PyTorch 2.x SDPA) help at longer sequences when shapes align to kernel requirements.

Relative position encodings (T5-style) bake distance into attention logits instead of adding vectors to embeddings; useful when extrapolating beyond train context. Dropout on attention weights (Transformer dropout) zeroes random edges in the softmax output, not the logits; keep `dropout=0.1` on attention and FFN in small-data runs. For IMDB-length text ($T \approx 512$), attention memory dominates; start with $T = 128$ subsample before scaling context. Stochastic depth (drop entire layers during training) is rare in small labs but explains why eval mode must be enabled before benchmarking val loss. Weight tying between embedding and output projection (in decoder-only LMs) cuts params; encoder-only classification labs use separate heads instead.

12 Debugging Checklist

- Loss not decreasing: wrong causal mask orientation (future tokens visible), or learning rate too high without warmup.
- All NaN after few steps: softmax over masked rows all $-\infty$; use large negative finite values instead of `-inf` in fp16.
- Position sensitivity absent: forgot positional encoding; model becomes bag-of-words.
- Slower than RNN on tiny T: transformer overhead dominates; expected for $T < 32$.
- Fine-tuned checkpoint garbage: tokenizer mismatch or post-norm vs pre-norm architecture flag wrong in config.
- Attention weights all uniform: missing $\sqrt{d_k}$ scaling or LayerNorm disabled by mistake in custom block.

13 Practice Problems

Problem 1. With $d_{\text{model}} = 512$, $H = 8$, confirm $d_k = 64$ and compute parameter count for Q/K/V/O projections in one MHA layer.

Problem 2. Hand-compute scaled dot-product attention for $T = 3$ with integer $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ given in class notes.

Problem 3. Implement a 2-layer encoder-only transformer for binary sequence classification on padded IMDB snippets.

Problem 4. Plot attention weights from head 0, layer 1 for a [CLS]-style pooling setup vs mean pooling.

Problem 5. Ablate positional encoding (zero it out) on a copy task with $T = 32$. Report accuracy vs baseline with sinusoidal positions.

Problem 6. Count total parameters for a 4-layer encoder ($d_{\text{model}} = 256$, $d_{ff} = 1024$) including embeddings and output head.

ALiBi adds linear distance bias to attention logits; compare extrapolation to $T = 256$ at inference vs sinusoidal positions trained at $T = 128$. Grouped-query attention (GQA) shares K/V heads across Q heads; not in the base lab but explains memory savings in modern LLM serving stacks. Encoder-decoder transformers add cross-attention sublayers; encoder-only blocks in **Module 09** omit causal mask entirely. Sandwich LayerNorm (norm inside FFN) stabilizes some deep stacks; always match `norm_first` flag to checkpoint architecture. Warmup steps should be 5 to 10% of total training steps for transformers trained from scratch on small copy tasks. Multi-head attention is single-head attention with split projections; H must divide d_{model} evenly.

Implement in **Lab 04**; compare bidirectional pre-training in **Module 09** and causal stacks in **Module 10**.

References

[1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.

[3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[4] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[5] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. Effective approaches to attention-based neural machine translation. In *EMNLP*, 2015.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.